L23 — Stack Operations: PUSH & POP; Parenthesis Matching

Unit: 2 | **Chapter:** Stacks & Recursion | **BT Level:** BT5 | **CO:** CO2, CO3

Learning Objectives

- Trace PUSH and POP operations step by step
- Implement the parenthesis matching algorithm using a stack
- Extend matching to multiple bracket types
- Apply stack-based validation to real programming scenarios

---

1. Detailed PUSH and POP Operations

1.1 PUSH (Add to Top)

```
Algorithm PUSH(stack, item):
  if stack.top == capacity - 1:
      raise Overflow
  stack.top ← stack.top + 1
  stack[stack.top] ← item
```

Trace: Push 5, 10, 15 into empty stack (capacity = 5):

Operation	top	Stack State
Initial	-1	[]
push(5)	0	[5]
push(10)	1	[5, 10]
push(15)	2	[5, 10, 15]

1.2 POP (Remove from Top)

```
Algorithm POP(stack):
  if stack.top == -1:
      raise Underflow
  item ← stack[stack.top]
  stack.top ← stack.top - 1
  return item
```

Trace continued:

Operation	top	Stack State	Returned
pop()	1	[5, 10]	15
pop()	0	[5]	10

pop()	-1	[]	5
pop()	—	—	Underflow!

---

2. Parenthesis / Bracket Matching

Problem: Given a string containing brackets `()`, `[]`, `{}`, determine if the brackets are properly matched and nested.

Valid: `()`, `()[]{}`, `{[()]}`, `((()))`
Invalid: `(]`, `([)]`, `{,)))`

2.1 Algorithm

```
Algorithm is_balanced(s):
  stack = empty stack
  for each char c in s:
      if c is an opening bracket: push(c)
      elif c is a closing bracket:
          if stack is empty: return False  (no matching opener)
          top = pop()
          if top and c don't match: return False
  return stack is empty   (all openers matched)
```

2.2 Implementation

```
def is_balanced(s):
    stack = []
    matching = {')': '(', ']': '[', '}': '{'}

    for char in s:
        if char in '([{':
            stack.append(char)
        elif char in ')]}':
            if not stack:
                return False              # Nothing to match
            if stack[-1] != matching[char]:
                return False              # Mismatched brackets
            stack.pop()

    return len(stack) == 0   # True if all openers were matched
```

2.3 Detailed Trace — `{ [ ( ) ] }`

Step	Char	Action	Stack
1	`{`	Push	`[{]`
2	`[`	Push	`[{, []`
3	`(`	Push	`[{, [, ()]`
4	`)`	Pop `(` — matches `)` ✓	`[{, []`

5	`]`	Pop `[` — matches `]` ✓ `[{]`
6	`}`	Pop `{` — matches `}` ✓ `[]`

Stack is empty → **Balanced ✓**

2.4 Trace — `([)]` (Invalid)

Step	Char	Action	Stack
1	`(`	Push	`[(]`
2	`[`	Push	`[(,` `[]`
3	`)`	Top is `[`, expected `(` → **mismatch!** —	

→ **Not balanced ✗**

---

3. Extended Applications

3.1 HTML/XML Tag Matching

```
def check_html_tags(html):
    """Check that HTML opening and closing tags are properly nested."""
    import re
    stack = []
    # Find all tags: <div>, </div>, <p>, etc.
    tags = re.findall(r'</?[a-zA-Z]+>', html)

    for tag in tags:
        if not tag.startswith('</'):
            # Opening tag — extract name and push
            tag_name = tag[1:-1]
            stack.append(tag_name)
        else:
            # Closing tag
            tag_name = tag[2:-1]
            if not stack or stack[-1] != tag_name:
                return False
            stack.pop()

    return len(stack) == 0

print(check_html_tags("<div><p></p></div>"))   # True
print(check_html_tags("<div><p></div></p>"))   # False
```

3.2 Minimum Bracket Additions to Make Valid

```
def min_additions_to_balance(s):
    """
    Returns minimum number of brackets to add/remove to balance s.
    """
```

```
    open_count = 0    # Unmatched '('
    close_count = 0   # Unmatched ')'

    for c in s:
        if c == '(':
            open_count += 1
        elif c == ')':
            if open_count > 0:
                open_count -= 1   # Matched
            else:
                close_count += 1  # Unmatched closing

    return open_count + close_count
```

3.3 Score of Parentheses (LeetCode #856)

```
def score_of_parentheses(s):
    """
    () = 1 point, (A) = 2*score(A), AB = score(A) + score(B)
    """
    stack = [0]   # Start with 0 (represents outer scope)
    for c in s:
        if c == '(':
            stack.append(0)
        else:  # c == ')'
            v = stack.pop()
            stack[-1] += max(2 * v, 1)
    return stack[0]

print(score_of_parentheses("()"))      # 1
print(score_of_parentheses("(())"))    # 2
print(score_of_parentheses("()()"))    # 2
print(score_of_parentheses("((()))"))  # 4
```

---

4. Complexity Analysis

Algorithm	Time	Space
Is Balanced	O(n)	O(n) worst case
Min Additions	O(n)	O(1)
Score of Parentheses	O(n)	O(n)

Each character is pushed/popped at most once → O(n) total operations.

---

5. Stack Size Analysis

The maximum stack size during parenthesis matching equals the **maximum nesting depth**:

- `(((((())))))` — depth 5 → max stack size 5

- `()()()()` — depth 1 → max stack size 1

For a string of length n, worst case stack size is n/2 (all opening brackets first) → O(n) space.

Summary

- **PUSH:** O(1) — increment top and place element.
- **POP:** O(1) — retrieve element and decrement top.
- Parenthesis matching: scan left to right, push openers, pop on closer and verify match.
- Valid if and only if: every closer matches its opener AND stack is empty at end.
- Time: O(n), Space: O(n) for the stack.
- Extends naturally to HTML validation, expression parsing, etc.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- LeetCode #20 (Valid Parentheses), #856 (Score of Parentheses)